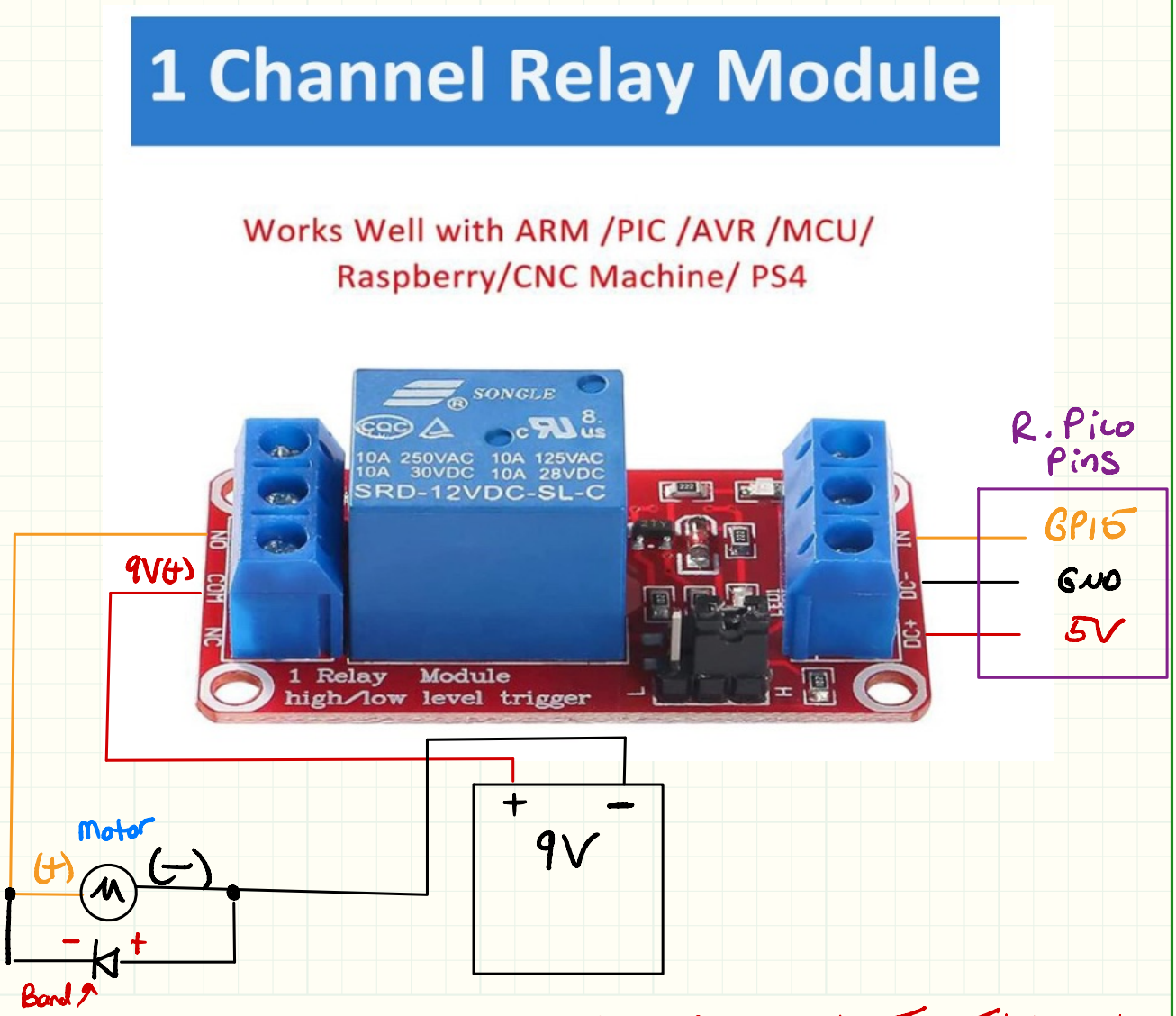
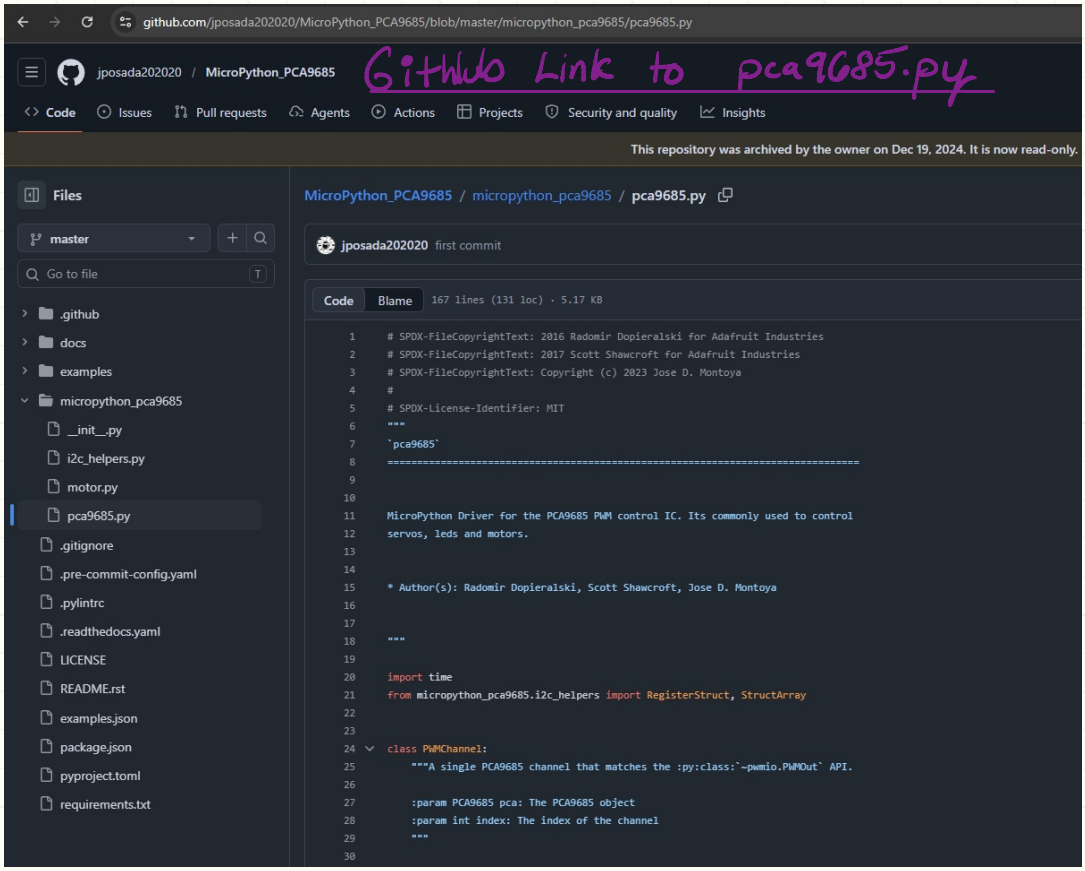
```python
# This script will ask the user to enter a value and turn a relay on or off based on that value
import machine

# First, declare the relay pin as a digital output
relay = machine.Pin(15, machine.Pin.OUT)

# This is the value at which the relay will turn on — anything above this kicks the relay on
THRESHOLD = 85

# Before we start our loop, make sure the relay starts in the OFF state
relay.value(0)
print("System Ready. Enter a value to control the relay...")

# Now, we are safe to enter our operation loop
while True:

    # Ask the user to enter a value and convert it from text to a number
    value = int(input("Enter a value (0 to 100): "))

    # If the value is above the threshold, turn the relay ON, otherwise turn it OFF
    if value > THRESHOLD:
        relay.value(1)
        print("Relay ON")
    else:
        relay.value(0)
        print("Relay OFF")
```



Fly Back Diode → Reduces EMF → Stops motor from Tripping Pico

```python
# This script will read from an analog joystick and control servo motors through a PCA9685 servo driver board
import machine
import time
from pca9685 import PCA9685

# First, declare the analog joystick pins (x = horizontal, y = vertical)
x_Joystick = machine.ADC(machine.Pin(26))

# Next, set up the I2C protocol so our Pico can talk to the PCA9685 board
# SCL is the clock line and SDA is the data line — both are required for I2C communication
i2c = machine.I2C(1, scl=machine.Pin(15), sda=machine.Pin(14), freq=400000)
pca = PCA9685(i2c)
pca.freq(50)  # Servo motors require a 50Hz PWM signal (one pulse every 20ms)

# This function will convert an angle (0 to 180 degrees) into a duty cycle value for the PCA9685
# The PCA9685 divides each 20ms period into 4096 steps, so we need to map our angle into that range
# A 500us pulse = 0 degrees, and a 2500us pulse = 180 degrees
def angle_to_duty_cycle(angle):
    angle = max(0, min(180, angle))      # Keep the angle within the safe 0 to 180 range
    return int((410 / 180) * angle + 102)

# This function will set a servo on a specific PCA9685 channel to a given angle (0 to 180)
def set_servo_to_angle(channel, angle):
    pca.duty(channel, angle_to_duty_cycle(angle))

# This function will convert a raw joystick ADC reading (0 to 65535) into a servo angle (0 to 180)
def joystick_to_servo_angle(joystick_value):
    return int((joystick_value / 65535) * 180)

# Before we start our loop, initialize the servos to their center position (90 degrees)
set_servo_to_angle(0, 90)   # X axis servo on channel 0

print("Servos Have Been Initialized. Starting Control Loop...")

# Now we are safe to enter our operation loop
while True:

    # Read the x-axis of the joystick, convert it to an angle, and move the servo on channel 0
    x_angle = joystick_to_servo_angle(x_Joystick.read_u16())
    set_servo_to_angle(0, x_angle)

    time.sleep_ms(50)  # Wait 50ms before reading the joystick again (runs 20 times per second)
```